

FINAL YEAR PROJECT REPORT

Project title:

**Design and development of a multi-agent system for
queue waiting-time estimation from video streams**

Prepared by:

Salmen Ben Ammar

Host Company:



Company Supervisor:

Ramzi Guesmi

Academic Supervisor:

Tarek Hamrouni

Academic Year 2025–2026



Dedications

To my parents,

No dedication could ever truly reflect my unending love and gratitude for all you've done for my education and well-being. May you recognize in this simple work the fulfillment of your dreams and the reward of your many sacrifices.

To my sister,

Your affection, support, and constant encouragement have been an immense source of motivation throughout my journey. Your belief in me helped me overcome challenges and stay focused on my goals. I wish you a life filled with success and happiness.

To my family,

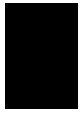
Your presence and unconditional love are my greatest treasures. Thank you for everything. You will forever remain my inexhaustible source of support, love, and comfort.

To all my friends,

My second family who made my journey an indescribable experience, and who have always encouraged me and believed in me. I wish you a future filled with achievement and joy.

To all of you

I dedicate this work.



Acknowledgements

At the conclusion of this project, I wish to express my sincere gratitude to the individuals who have greatly contributed to its successful completion.

First, I would like to express my profound appreciation to the entire team at **Data-DoIt** for graciously providing me the opportunity to carry out this final-year project.

I would also like to extend my sincere thanks to my professional supervisor, **Mr. Ramzi Guesmi**, for his unwavering support, constant availability, patience, and his relevant and constructive guidance, which proved invaluable throughout this project.

I warmly thank my academic supervisor, **Mr. Tarek Hamrouni**, for his constant availability, patience, and insightful advice, which were essential for the successful completion of this work. His guidance, encouragement, and constructive feedback not only helped me overcome numerous challenges but also inspired me to strive for excellence and deepen my understanding throughout this project.

I would also like to take this opportunity to express my gratitude to the entire faculty of **Sesame University**, whose contributions have been fundamental to my education over the past three years.

Finally, I sincerely thank the members of the jury for agreeing to evaluate my work. Their expertise and commitment are greatly appreciated, and I wish to express my deep gratitude for their contribution to the assessment of this project.



Table of Content

List of Figures	ix
List of Tables	x
General Introduction	xi
1 Context of the Project	1
1.1 Introduction	2
1.2 Host Organization	2
1.2.1 DataDoIt	2
1.2.2 Sectors of Activity	3
1.3 Presentation of the Project	4
1.3.1 Project Context	4
1.3.2 Problem Statement	4
1.3.3 Study of Existing Approaches	5
1.3.4 Proposed Solution	6
1.3.5 Objectives of the Project	7
1.3.6 Project Scope and Boundaries	7
1.3.7 Target Audience	8
1.4 Project Management Methodology	9
1.4.1 Choice of Methodology: Agile	9
1.4.2 SCRUM	9
1.4.2.1 Module A: Backend Core (Sprints 1-3)	9
1.4.2.2 Module B: Web Experience (Sprints 4-5)	9

1.4.2.3	Module C: Automation and Finalization (Sprints 6-8) . .	10
1.5	Conclusion	10
2	Requirements Specification and Analysis	12
2.1	Introduction	13
2.2	Requirements Specification	13
2.2.1	Identification of Actors	13
2.2.2	Functional Requirements	13
2.2.3	Non-functional Requirements	14
2.3	Product Backlog	16
2.4	Global Use Case Diagram	17
2.5	Sprint Scope Overview	18
2.6	Data Modeling	19
2.6.1	Core Entities	19
2.6.2	Data Lifecycle and Persistence Strategy	20
2.7	Project Environment	20
2.7.1	Hardware and Software Environment	20
2.7.2	Technologies Used	21
2.8	Software Architecture	21
2.8.1	Overall Architecture	21
2.8.2	Backend Architecture	22
2.8.3	Frontend Architecture	22
2.9	Conclusion	23
3	Sprint 1: Foundation and Perception	24
3.1	Introduction	25
3.2	Sprint Backlog	25
3.3	Sprint Analysis	25

3.3.1	Detailed Use Case Diagram	25
3.3.2	Use Case Scenarios	26
3.4	Design	26
3.4.1	Activity Diagram	26
3.4.2	Sequence Diagrams	27
3.5	Implementation	27
3.5.1	Detection Layer	27
3.5.2	Tracking Layer	28
3.5.3	Zone-Aware Filtering	28
3.5.4	Main Loop Integration	28
3.6	Conclusion	29
4	Sprint 2: Analytics and Uncertainty	30
4.1	Introduction	31
4.2	Sprint Backlog	31
4.3	Design	31
4.3.1	Component Diagram	31
4.3.2	Data Flow Diagram	31
4.3.3	Sequence Diagrams	32
4.4	Implementation	33
4.4.1	Queue Event Windows and Rate Estimation	33
4.4.2	Wait-Time Logic and Stability Handling	33
4.4.3	Bayesian and Variance-Based Uncertainty	34
4.4.4	Uncertainty Classification and Runtime Contract	34
4.5	Conclusion	34
5	Sprint 3: Advanced Connectivity and API	35
5.1	Introduction	36

5.2	Sprint Backlog	36
5.3	Design	36
5.3.1	Component Diagram	36
5.3.2	Data Flow Diagram	37
5.3.3	Sequence Diagrams	37
5.4	Implementation	38
5.4.1	RTSP Source Integration	38
5.4.2	ONVIF Discovery and Stream Resolution	38
5.4.3	FastAPI BFF and Typed Contract	38
5.4.4	Runtime Orchestration and Real-Time Events	39
5.4.5	Persistence and Recovery	39
5.5	Conclusion	39
6	Sprint 4: Web Design and Scaffolding	40
6.1	Introduction	41
6.2	Sprint Backlog	41
6.3	Design	41
6.3.1	UI and UX Design	41
6.3.2	Frontend Architecture	42
6.4	Implementation	42
6.5	Conclusion	43
7	Sprint 5: Frontend Integration and Live Monitoring	44
7.1	Introduction	45
7.2	Sprint Backlog	45
7.3	Design	45
7.3.1	Data Flow Diagram	45
7.3.2	Sequence Diagrams	45

7.4	Implementation	46
7.4.1	Typed API Contract Integration	46
7.4.2	Dashboard State Orchestration	46
7.4.3	WebSocket Reliability and Fallback Logic	47
7.4.4	Dashboard UI Integration	47
7.4.5	Integration Testing	47
7.5	Conclusion	48
8	Sprint 6: n8n Automation and Alerting	49
8.1	Introduction	50
8.2	Sprint Backlog	50
8.3	Design	50
8.3.1	Component Diagram	50
8.3.2	Data Flow Diagram	50
8.4	Implementation	51
8.4.1	Webhook Payload Contract	51
8.4.2	Reliable Webhook Delivery	52
8.4.3	Runtime Alert Dispatch Integration	52
8.4.4	n8n Workflow Routing and Telegram Delivery	52
8.4.5	Automation Testing Artifacts	53
8.5	Conclusion	53
9	Sprint 7: Evaluation and Optimization	54
9.1	Introduction	55
9.2	Sprint Backlog	55
9.3	Design	55
9.3.1	Test Plan	55
9.3.2	Performance Metrics	56

9.4	Implementation	56
9.4.1	Automated Test Coverage	56
9.4.2	Runtime Optimization Actions	56
9.4.3	Resilience Improvements	57
9.4.4	Observed Outcome	57
9.5	Conclusion	57
10	General Conclusion	58
10.1	Summary	59
10.2	Objective Review	59
10.3	Future Work	60
A	Appendices	62
A.1	Useful Commands	62
A.2	Example Webhook Payload	62
A.3	Items to Add Before Final Submission	63



List of Figures

2.1	Global use case diagram of the queue wait-time estimation system. . . .	17
3.1	Sprint 1 use case diagram for detection and tracking in queue zones. . .	26
3.2	Sprint 1 activity flow from frame ingestion to queue-zone updates. . . .	27
4.1	Sprint 2 components for queue analysis and uncertainty computation. .	32
4.2	Data flow from tracked events to queue metrics and confidence intervals.	32
5.1	Sprint 3 components for camera integration and backend services. . . .	37
5.2	Data flow from video feeds to API responses and WebSocket broadcasts.	37
6.1	Frontend architecture for routing, components, and state boundaries. . .	42
7.1	Data flow between backend services and frontend monitoring components.	46
8.1	Components involved in backend-to-n8n-to-Telegram automation flow.	51
8.2	Alert payload flow from backend metrics to external notifications. . . .	51



List of Tables

1.1	Collaboration perimeter with DataDoIt	3
1.2	From operational pain points to system-level requirements	5
1.3	Positioning of the proposed solution against common approach families	6
1.4	Objective-to-module traceability	8
1.5	Scrum module mapping used in this report	10
2.1	Actor responsibility matrix	14
2.2	Functional requirement traceability matrix	15
2.3	Non-functional requirement quality matrix	16
2.4	Backlog domain structuring	17
2.5	Primary global use cases	18
2.6	Project sprint scope overview	18
2.7	Conceptual entity mapping	19
2.8	Execution environment profile	20
2.9	Architecture layer-to-module mapping	22



General Introduction

In a context where digital transformation and artificial intelligence increasingly shape operational decision-making, service organizations are under strong pressure to reduce waiting time and improve service quality. Supermarkets, banks, and service centers face the same recurring challenge: queues are dynamic, difficult to monitor manually, and highly sensitive to demand fluctuations.

In practice, customers often join a queue without a clear idea of expected waiting time, while managers and operational staff do not always have reliable real-time indicators to adapt resources quickly. Manual observation remains subjective and costly, while simple counting solutions do not provide robust waiting-time estimation with confidence levels.

This end-of-studies project addresses this gap through the design and development of an intelligent real-time queue monitoring system based on computer vision and queueing theory, with explicit uncertainty quantification. The proposed system detects and tracks people from video streams, analyzes occupancy inside configurable queue zones, estimates arrival and service rates, computes waiting time, and exposes confidence-aware metrics for operational use.

This work combines classical queueing theory with modern computer-vision detection and tracking techniques [1], [2] to provide actionable waiting-time estimates with quantified uncertainty.

This report is organized as follows. Chapter 1 presents the context of the project, host organization, problem statement, and objectives. Chapter 2 defines requirements specification and analysis, including architecture choices. Chapters 3 to 9 present sprint-

by-sprint implementation progress from perception foundations to evaluation and optimization. Chapter 10 provides the general conclusion and future work directions.

Context of the Project

Table of Content

1.1	Introduction	2
1.2	Host Organization	2
1.2.1	DataDoIt	2
1.2.2	Sectors of Activity	3
1.3	Presentation of the Project	4
1.3.1	Project Context	4
1.3.2	Problem Statement	4
1.3.3	Study of Existing Approaches	5
1.3.4	Proposed Solution	6
1.3.5	Objectives of the Project	7
1.3.6	Project Scope and Boundaries	7
1.3.7	Target Audience	8
1.4	Project Management Methodology	9
1.4.1	Choice of Methodology: Agile	9
1.4.2	SCRUM	9
1.5	Conclusion	10

1.1 Introduction

This chapter positions the Queue Wait-Time Estimation System in its operational, organizational, and methodological context. Beyond a general introduction, it clarifies why queue monitoring remains a high-impact problem for service organizations and why an end-to-end technical solution is required instead of isolated counting tools.

The chapter also establishes the project baseline used in the rest of the report: collaboration context with DataDoIt, problem formalization, comparison with existing approaches, solution scope, and implementation governance through Scrum modules. This framing is intentionally detailed because later technical chapters depend on these assumptions.

1.2 Host Organization

1.2.1 DataDoIt

This end-of-studies project was carried out in collaboration with DataDoIt, a data-driven sustainability consulting firm headquartered at Elghazal Technopark in Tunis, Tunisia. DataDoIt specializes in transforming environmental and sustainability commitments into measurable, actionable outcomes for governments, municipalities, and enterprises across the EMEA region. The company operates at the intersection of data science and sustainability, offering services that span decarbonization strategy, ESG reporting, smart resource analytics, and circular economy implementation. Its approach, summarized as Measure It, Optimize It, Do It, reflects a methodology grounded in quantitative assessment, strategic planning, and hands-on technical deployment. To date, the company has delivered over 15 active projects across 6 key sectors, with a reported average ROI of 2.3x and a 94% client retention rate.

The collaboration provided a concrete engineering context in which data collection,

real-time monitoring, and automated analytics are central concerns, requirements that directly shaped the design and scope of this project.

Table 1.1: Collaboration perimeter with DataDoIt

Dimension	Scope in this project
Business context	Queue management in service scenarios where waiting time and staffing decisions are operationally sensitive.
Engineering constraints	Multi-source video ingestion, variable visual conditions, low-latency monitoring, and integration with automation channels.
Expected outputs	Actionable queue indicators, uncertainty-aware wait-time estimation, and alert-ready payloads for external workflows.
Validation mode	Iterative sprint reviews using implementation evidence from backend modules, API contracts, frontend behavior, and automation tests.

1.2.2 Sectors of Activity

DataDoIt delivers sustainability solutions across six mission-critical sectors where data-driven environmental performance has direct operational and financial implications:

- government and public sector institutions seeking to translate climate policy into measurable action,
- smart cities and municipalities requiring integrated urban resilience and climate adaptation systems,
- industrial and manufacturing operations targeting Scope 1-3 carbon footprint reduction and supply chain efficiency,
- real estate and infrastructure organizations pursuing green certification and climate-resilient asset development,
- financial services firms navigating ESG compliance and green finance opportunities,

- energy and utilities providers accelerating the transition toward renewable integration and grid modernization.

1.3 Presentation of the Project

1.3.1 Project Context

Manual queue supervision is often subjective, difficult to scale, and highly dependent on operator availability. In many environments, monitoring remains reactive: teams observe congestion after it appears rather than anticipating it early with measurable indicators.

This project addresses that gap by transforming camera streams into decision-oriented telemetry. The target output is not only occupancy counts, but a coherent set of operational metrics: people in zone, arrival rate, service rate, expected wait time, uncertainty level, and alert states consumable by monitoring and automation layers.

1.3.2 Problem Statement

Existing operational practices and basic counting tools present recurring limitations:

- they provide limited anticipation of congestion,
- they rarely estimate waiting time explicitly,
- they do not quantify uncertainty, which is critical in noisy visual environments,
- they are often disconnected from automated response workflows.

As a consequence, interventions (staff reallocation, lane opening, escalation) can be delayed or triggered on incomplete signals. The problem is therefore both analytical and integrational: producing robust indicators and delivering them through actionable channels.

Table 1.2: From operational pain points to system-level requirements

Observed pain point	Operational impact	Required system capability
Manual visual estimation is inconsistent across shifts.	Queue state interpretation varies between operators, reducing decision reliability.	Standardized per-feed metrics with explicit formulas and stable update cadence.
Occupancy-only monitoring is descriptive but not predictive.	Teams react late because no forward-looking wait-time signal is available.	Continuous estimation of λ , μ , and expected wait time W .
No confidence indicator accompanies reported metrics.	High-noise periods can produce overconfident decisions.	Uncertainty quantification with interval bounds and qualitative confidence level.
Alerting tools are not integrated with runtime analytics.	Escalation depends on manual relay, increasing response latency.	Automated webhook dispatch and workflow-based notification routing.

1.3.3 Study of Existing Approaches

Recent computer-vision frameworks provide strong building blocks for person perception, notably YOLO-based detection and tracking algorithms such as ByteTrack [2], [3]. Queueing theory, in parallel, offers formal models for service-system behavior, including interpretable relationships between arrival rate, service rate, and expected waiting time [1].

However, many practical deployments remain partial. Some focus on counting without identity continuity, others on visualization without uncertainty semantics, and others on analytics without operational delivery channels. In addition, integration boundaries between perception pipelines, web supervision, and alert automation are frequently treated as separate projects instead of one coherent architecture.

Table 1.3: Positioning of the proposed solution against common approach families

Approach family	Strengths	Typical limitation
Manual or occupancy-only monitoring	Simple to deploy and easy to interpret at first glance.	Weak predictive capability and high operator subjectivity.
Vision detection without persistent tracking	Fast frame-level counting.	Identity discontinuity can distort arrival and departure logic.
Analytical queue models without vision coupling	Strong mathematical interpretability.	Requires reliable event extraction pipeline from real scenes.
Proposed integrated pipeline	Couples perception, queue analytics, uncertainty, web monitoring, and automation.	Requires careful multi-layer integration and validation.

1.3.4 Proposed Solution

The proposed system combines perception, analytics, and integration in one modular pipeline:

1. person detection and multi-object tracking from camera feeds,
2. polygon-based queue-zone filtering,
3. estimation of λ , μ , and expected wait time,
4. Bayesian uncertainty quantification for key metrics,

5. real-time web supervision and automation through n8n webhooks and Telegram alerts.

This architecture aims to convert visual observations into operational actions while keeping each layer independently testable. Detection/tracking modules can evolve without rewriting queue formulas; web and automation channels can evolve without changing the core metric contract.

1.3.5 Objectives of the Project

The project objectives are to:

1. detect and track people in real time,
2. count people in configurable queue zones,
3. estimate arrival and service rates,
4. compute expected wait time,
5. quantify uncertainty for operational reliability,
6. expose metrics through web interfaces and automation workflows.

To keep implementation traceable, objectives are mapped to delivery modules and observable outputs.

1.3.6 Project Scope and Boundaries

The implementation scope includes backend perception and analytics modules, API/runtime integration, web monitoring, and external workflow automation. The report deliberately focuses on this web-first operational architecture.

All implementation details and validation narratives are presented through the backend, web, and automation delivery tracks to maintain a consistent production-oriented scope.

Table 1.4: Objective-to-module traceability

Objective ID	Objective	Primary module	Observable output
OBJ-01	Real-time detection and tracking	Module A	Stable in-frame detections with persistent tracker IDs.
OBJ-02	Queue-zone counting	Module A	Per-frame in-zone people count and zone-aware overlays.
OBJ-03	Queue metric estimation (λ, μ, W)	Module A	Continuous metric payloads from queue analysis pipeline.
OBJ-04	Uncertainty quantification	Module A	Confidence intervals and uncertainty level labels.
OBJ-05	Web monitoring	Module B	Live dashboard with feed controls, KPIs, and stream views.
OBJ-06	Automation integration	Module C	Webhook-based alert routing through n8n and Telegram.

1.3.7 Target Audience

The system targets three stakeholder groups with complementary goals:

- **Operators:** need low-latency visibility of queue status and actionable alert signals.
- **Managers:** need interpretable indicators for staffing and service-quality planning.
- **End users:** benefit indirectly from reduced waiting times and smoother service flow.

This separation is important because each group consumes different levels of abstraction, from per-feed live events to aggregated performance trends.

1.4 Project Management Methodology

1.4.1 Choice of Methodology: Agile

An Agile approach was selected to deliver functional increments quickly while reducing integration risks early. The project combines multiple technical domains (computer vision, queue analytics, backend APIs, frontend workflows, and automation), each with uncertainties that are difficult to resolve using a single monolithic delivery cycle.

Iterative delivery made it possible to validate contracts progressively: first perception outputs, then analytical reliability, then API/runtime integration, then web usability, and finally workflow automation and hardening.

1.4.2 SCRUM

Development followed sprint-based Scrum planning with iterative validation. For report readability and implementation traceability, work is grouped into three logical modules.

1.4.2.1 Module A: Backend Core (Sprints 1-3)

This module established the vision and analytics backbone:

- Sprint 1: detection, tracking, and zone filtering foundation,
- Sprint 2: queue metrics and uncertainty modeling,
- Sprint 3: camera connectivity and backend API exposure.

1.4.2.2 Module B: Web Experience (Sprints 4-5)

This module focused on the web supervision experience:

- Sprint 4: web design system and frontend scaffolding,

- Sprint 5: frontend integration with live backend metrics.

1.4.2.3 Module C: Automation and Finalization (Sprints 6-8)

This module covered integration hardening and closure:

- Sprint 6: n8n automation and Telegram alerting,
- Sprint 7: evaluation and optimization,
- Sprint 8: report finalization and project consolidation.

Table 1.5: Scrum module mapping used in this report

Module	Sprints	Main focus	Validation perspective
Module A: Backend Core	1 to 3	Vision pipeline, queue analytics, camera connectivity, API services	Technical correctness of perception and metrics contracts.
Module B: Web Experience	4 to 5	Frontend design system, dashboard scaffolding, live integration	Usability, interaction continuity, and live data rendering stability.
Module C: Automation and Finalization	6 to 8	n8n alerting, optimization, validation, and reporting	End-to-end operability, alert reliability, and delivery readiness.

1.5 Conclusion

This chapter positioned the project in its organizational and technical context, formalized the addressed operational problem, and justified the selected delivery strategy. Chapter

2 builds on this baseline by formalizing requirements, architecture constraints, and analysis artifacts used to guide implementation decisions.

Table of Content

2.1	Introduction	13
2.2	Requirements Specification	13
2.2.1	Identification of Actors	13
2.2.2	Functional Requirements	13
2.2.3	Non-functional Requirements	14
2.3	Product Backlog	16
2.4	Global Use Case Diagram	17
2.5	Sprint Scope Overview	18
2.6	Data Modeling	19
2.6.1	Core Entities	19
2.6.2	Data Lifecycle and Persistence Strategy	20
2.7	Project Environment	20
2.7.1	Hardware and Software Environment	20
2.7.2	Technologies Used	21
2.8	Software Architecture	21
2.8.1	Overall Architecture	21
2.8.2	Backend Architecture	22
2.8.3	Frontend Architecture	22
2.9	Conclusion	23

2.1 Introduction

This chapter formalizes the requirement baseline that guided implementation choices across backend perception, queue analytics, web supervision, and automation workflows. Its role is twofold: define what the system must provide, and justify how architectural decomposition supports those requirements.

The chapter therefore goes beyond feature listing. It introduces requirement traceability, actor responsibilities, quality constraints, sprint-scope mapping, conceptual data entities, and software architecture layers. These elements are used as reference contracts for the sprint implementation chapters.

2.2 Requirements Specification

2.2.1 Identification of Actors

The system involves three primary actors:

- **Administrator:** configures sources, thresholds, and integration parameters.
- **Operator:** monitors live queues and reacts to alerts during operations.
- **Manager:** analyzes historical trends and uses indicators for planning decisions.

Although these roles interact with the same platform, they consume different abstractions. Administrators require configuration and diagnostics depth, operators require low-latency situational awareness, and managers require interpretable KPI summaries for decision support.

2.2.2 Functional Requirements

The system must provide the following core functionalities:

- real-time person detection and tracking,

Table 2.1: Actor responsibility matrix

Actor	Main responsibilities	Typical interactions
Administrator	Configure feeds, zones, threshold rules, and integration credentials.	Uses setup and settings flows, validates connectivity, and supervises runtime health signals.
Operator	Monitor queue status in real time and react to warnings/critical alerts.	Uses dashboard KPIs, live feed views, and alert panels for immediate response.
Manager	Review aggregate behavior and support planning decisions.	Consumes historical trends, uncertainty-aware metrics, and performance summaries.

- configurable queue zone definition,
- estimation of arrival rate (λ), service rate (μ), and wait time (W),
- uncertainty quantification for queue metrics,
- real-time web dashboard updates and alerts,
- RTSP and ONVIF camera integration,
- automated webhook and Telegram notifications,
- historical data logging and analytics.

To support verification and sprint-level accountability, these requirements are formalized in a traceability matrix.

2.2.3 Non-functional Requirements

The main quality constraints are:

- **Performance:** maintain real-time behavior and low alert latency.
- **Scalability:** support multiple feeds and independent feed lifecycle control.
- **Reliability:** tolerate transient source failures and integration interruptions.
- **Usability:** provide clear web workflows for monitoring and configuration.

Table 2.2: Functional requirement traceability matrix

ID	Requirement statement	Priority	Verification strategy
FR-01	Detect people from live or recorded video streams in near real time.	High	Validate detector outputs and runtime overlays from <code>backend/src/detector.py</code> integration.
FR-02	Maintain identity continuity through tracking for event-level queue analysis.	High	Validate tracker ID continuity from <code>backend/src/tracker.py</code> in runtime scenarios.
FR-03	Restrict counting to configurable polygon queue zones.	High	Validate zone inclusion logic from <code>backend/src/zone_manager.py</code> .
FR-04	Compute λ , μ , and expected wait time from event windows.	High	Validate queue formulas and stability handling in <code>backend/src/queue_analyzer.py</code> .
FR-05	Compute uncertainty bounds and qualitative confidence levels.	High	Validate interval outputs and fallback behavior in <code>backend/src/uncertainty.py</code> .
FR-06	Expose feed control and live updates through API and websocket channels.	High	Validate endpoint and runtime behavior through API and integration tests.
FR-07	Dispatch alert payloads to external automation workflows.	Medium	Validate webhook dispatch and n8n branch behavior using script-based scenarios.
FR-08	Persist operational metrics and metadata for analysis and auditing.	Medium	Validate CSV and SQLite paths through runtime execution and persistence checks.

- **Security:** protect integration channels and service endpoints.
- **Maintainability:** keep modular code organization and testable interfaces.

These quality attributes are treated as engineering contracts, not only design intentions.

Table 2.3: Non-functional requirement quality matrix

ID	Quality attribute	Target indicator	Validation approach
NFR-01	Performance	Stable near-real-time loop and bounded alert propagation delay.	Runtime observation under representative feed scenarios.
NFR-02	Scalability	Independent lifecycle control for multiple feed contexts.	Feed registry behavior and state-transition verification.
NFR-03	Reliability	Graceful handling of camera drops and webhook transmission failures.	Reconnect paths and retry/fallback test scenarios.
NFR-04	Usability	Clear setup-to-monitoring flow for non-developer users.	Dashboard integration tests and workflow walkthroughs.
NFR-05	Security	Controlled integration access via shared secret and credential isolation.	Webhook secret validation and credentialized channel checks.
NFR-06	Maintainability	Modular architecture with explicit contracts and targeted tests.	Code organization review and test-suite coverage by module.

2.3 Product Backlog

The product backlog was prioritized by business value and technical dependency. For implementation governance, backlog items were grouped into three domains:

- **Backend domain:** perception pipeline, queue analytics, API and stream interfaces.
- **Web domain:** dashboard UX, live monitoring integration, zone editing workflows.
- **Automation domain:** n8n workflow integration, alert routing, webhook reliability.

Early execution emphasized backend foundations to reduce downstream integration risk. This sequencing ensured that web and automation layers consumed a stable metric

contract rather than moving targets.

Table 2.4: Backlog domain structuring

Domain	Representative backlog scope	Dependency rationale
Backend	Detection/tracking, zone logic, queue formulas, uncertainty, feed runtime services.	Provides source-of-truth metrics consumed by all upper layers.
Web	Dashboard pages, typed API integration, websocket synchronization, feed controls.	Depends on stable backend contracts and event schemas.
Automation	Webhook payload contract, routing, cooldown, external notifications, archival.	Depends on reliable alert semantics and validated thresholds.

2.4 Global Use Case Diagram

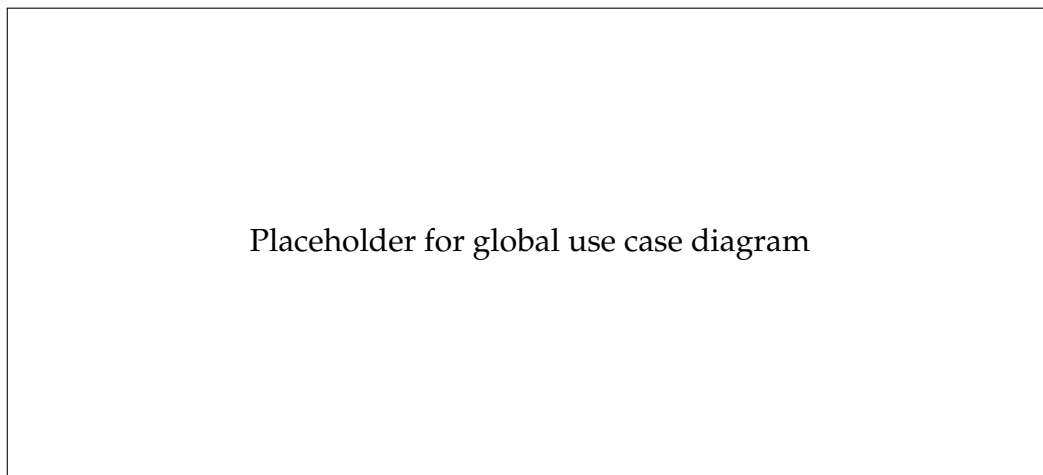


Figure 2.1: Global use case diagram of the queue wait-time estimation system.

At system level, use cases can be organized into four interaction groups: configuration, monitoring, analytics interpretation, and alert handling. The global diagram should reflect these groups and their actor responsibilities.

Table 2.5: Primary global use cases

Use case ID	Use case summary	Primary actor
UC-01	Register and configure a video feed source.	Administrator
UC-02	Define or update queue zone geometry.	Administrator
UC-03	Monitor live queue metrics and feed status.	Operator
UC-04	Receive and interpret warning or critical alerts.	Operator
UC-05	Review trends and performance indicators.	Manager
UC-06	Validate automation workflow behavior for alerts.	Administrator

2.5 Sprint Scope Overview

The project implementation is organized into eight sprints. Table 2.6 summarizes sprint scope and expected outcomes without using calendar-based timeline tracking.

Table 2.6: Project sprint scope overview

Sprint	Main outcome	Representative validation lens
Sprint 1	Detection, tracking, and queue-zone perception foundation	Perception correctness and tracker continuity
Sprint 2	Queue analytics and uncertainty quantification	Formula validity and interval consistency
Sprint 3	Camera connectivity and backend API exposure	Runtime control and contract stability
Sprint 4	Web design system and frontend scaffolding	UX structure and component baseline
Sprint 5	Frontend integration and live monitoring	API/websocket synchronization quality
Sprint 6	n8n automation and Telegram alerting	Alert routing and delivery reliability
Sprint 7	Evaluation and optimization	Test depth and runtime hardening
Sprint 8	Final consolidation, reporting, and delivery	End-to-end consistency and documentation quality

2.6 Data Modeling

The data model combines runtime monitoring entities and persistence entities. Its design objective is to support both low-latency supervision and post-hoc analysis with minimal contract drift across modules.

2.6.1 Core Entities

The main objects are:

- **VideoFeed**: source identity, connectivity, runtime status, and configuration.
- **QueueMetrics**: timestamped indicators (people in zone, λ , μ , W , stability).
- **Alert**: severity, threshold context, and notification payload.
- **Establishment** and **Caisse**: business metadata used to organize feeds.

Table 2.7: Conceptual entity mapping

Entity	Core semantics	Producer/consumer context
VideoFeed	Declares source type, identity, status, and runtime control metadata.	Produced by onboarding/configuration flows; consumed by runtime and dashboard layers.
QueueMetrics	Captures analytical outputs used for monitoring and decision support.	Produced by analysis loop; consumed by web-socket/API/dashboard and alert logic.
Alert	Encodes threshold breach context and notification-ready payload content.	Produced by threshold evaluation; consumed by webhook automation and archival paths.
Establishment / Caisse	Binds feeds to business-level organization context.	Produced by metadata management APIs; consumed by setup and filtering interfaces.

2.6.2 Data Lifecycle and Persistence Strategy

Runtime metrics are generated continuously and distributed through live channels, while selected records are persisted for auditability and trend analysis. This dual-path strategy (stream plus persistence) improves resilience: temporary integration failures do not imply complete data loss.

CSV logging supports rapid observability and portability, while SQLite supports structured metadata and alert archival. Together, these storage paths enable both operational continuity and post-run diagnosis.

2.7 Project Environment

2.7.1 Hardware and Software Environment

The development workflow combines a Python backend environment, a TypeScript frontend environment, and containerized automation services. The project is designed to run on standard workstations and can exploit hardware acceleration when compatible GPU resources are available.

Table 2.8: Execution environment profile

Environment layer	Main tooling	Purpose
Backend runtime	Python ecosystem with vision and scientific libraries	Perception, queue analytics, uncertainty, and integration dispatch.
Frontend runtime	Node/Bun-based TypeScript toolchain with Vite	Dashboard rendering, interaction workflows, and live-state synchronization.
Automation runtime	Self-hosted n8n with webhook endpoints	Alert routing, cooldown logic, and external notifications.
Persistence/runtime support	CSV outputs and SQLite metadata store	Historical analysis and system state continuity.

2.7.2 Technologies Used

The selected technology stack is:

- **Vision pipeline:** YOLO26, ByteTrack, OpenCV [2], [3], [4].
- **Queue analytics:** NumPy, Pandas, SciPy, M/M/1 modeling [1].
- **Backend API:** FastAPI and Pydantic v2.
- **Frontend:** React, TypeScript, Vite, Tailwind CSS.
- **Automation:** n8n workflows with Telegram notifications.
- **Persistence:** CSV metrics logging and SQLite metadata.

Technology selection follows a modular criterion: each component must provide clear contract boundaries and practical interoperability with adjacent layers.

2.8 Software Architecture

2.8.1 Overall Architecture

The system architecture is organized in four layers:

1. **Video acquisition:** local files, RTSP streams, and ONVIF discovery.
2. **Perception:** detection, tracking, and queue-zone filtering.
3. **Analysis:** queue metrics estimation and uncertainty modeling.
4. **Output and integration:** web dashboard, webhook dispatch, and alert automation.

This layering constrains coupling and simplifies testing. For example, queue formulas depend on perception events, but do not depend on frontend rendering details.

Table 2.9: Architecture layer-to-module mapping

Layer	Responsibilities	Representative modules
Video acquisition	Source opening, stream transport handling, reconnect behavior.	<code>video_capture.py</code> , <code>rtsp_camera.py</code> , <code>onvif_client.py</code>
Perception	Person detection, tracking continuity, zone-aware filtering.	<code>detector.py</code> , <code>tracker.py</code> , <code>zone_manager.py</code>
Analysis	Event-window rates, wait-time logic, uncertainty bounds.	<code>queue_analyzer.py</code> , <code>uncertainty.py</code> , <code>threshold_detector.py</code>
Output/integration	API/websocket exposure, webhook dispatch, persistence, dashboard consumption.	<code>api/app.py</code> , <code>api/runtime.py</code> , <code>webhook_client.py</code> , frontend hooks/pages

2.8.2 Backend Architecture

The backend exposes REST and websocket interfaces for feed lifecycle control and live metrics. It includes modules for detector orchestration, tracking, queue analysis, threshold detection, webhook delivery, and camera-source management.

Runtime orchestration is centered around feed-level lifecycle control and event broadcasting. This design supports independent feed state transitions and clear event contracts consumed by the dashboard and automation pipeline.

Persistence and integration are treated as first-class concerns: CSV metrics logging, SQLite metadata state, and webhook dispatch reliability are all part of the operational architecture rather than optional add-ons.

2.8.3 Frontend Architecture

The frontend follows a component-based architecture with typed API contract mapping, feed-oriented dashboard composition, and websocket-driven synchronization. It separates data-access concerns (API client and hooks) from presentation concerns (pages

and reusable components).

This separation improves maintainability: backend contract adjustments can be localized to integration hooks and API utilities, while visual components remain focused on rendering and interaction logic.

2.9 Conclusion

This chapter established the detailed requirement baseline and architecture rationale used throughout implementation. The next chapters translate these contracts into sprint-by-sprint technical realization, from perception foundations to integration, automation, and validation outcomes.

Sprint 1: Foundation and Perception

Table of Content

3.1	Introduction	25
3.2	Sprint Backlog	25
3.3	Sprint Analysis	25
3.3.1	Detailed Use Case Diagram	25
3.3.2	Use Case Scenarios	26
3.4	Design	26
3.4.1	Activity Diagram	26
3.4.2	Sequence Diagrams	27
3.5	Implementation	27
3.5.1	Detection Layer	27
3.5.2	Tracking Layer	28
3.5.3	Zone-Aware Filtering	28
3.5.4	Main Loop Integration	28
3.6	Conclusion	29

3.1 Introduction

Sprint 1 established the perception backbone of the system: person detection, multi-object tracking, and zone-aware counting from video streams. The implementation is centered on the backend runtime pipeline in `backend/src/main.py`, with dedicated modules for detection, tracking, and polygon-zone logic.

3.2 Sprint Backlog

The committed Sprint 1 backlog focused on the minimum viable perception chain:

- Integrate YOLO-based person detection with configurable model size and inference device.
- Integrate persistent tracking so each detected person keeps a stable ID across frames.
- Define queue zones as polygons and count only detections inside these zones.
- Build a real-time loop that connects capture, detection, tracking, zone filtering, and rendering.
- Expose runtime controls through CLI options for source type, model parameters, and processing stride.

3.3 Sprint Analysis

3.3.1 Detailed Use Case Diagram

The use case is centered on the operator configuring a source and zone, then launching continuous analysis. For each frame, the system detects persons, associates identities,

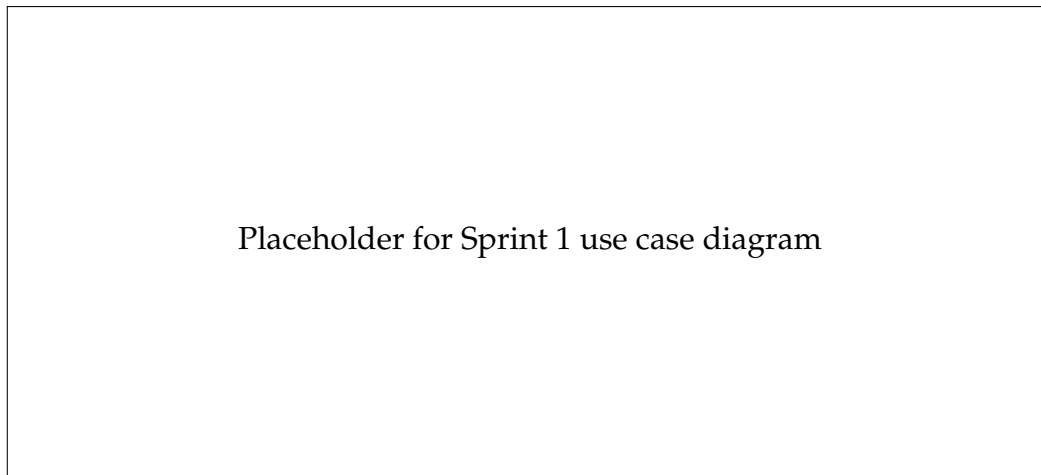


Figure 3.1: Sprint 1 use case diagram for detection and tracking in queue zones.

evaluates zone membership, and updates the queue state. This sequence is the basis for all later analytics and automation layers.

3.3.2 Use Case Scenarios

Three key scenarios are implemented in Sprint 1:

1. **Person entry into queue zone:** a new tracker ID appears inside the polygon and is treated as a queue arrival candidate.
2. **Tracker continuity across frames:** the same person is followed through consecutive frames using ByteTrack, reducing duplicate counting.
3. **Zone filtering under mixed scenes:** detections outside the polygon are ignored for queue metrics, preventing unrelated people from polluting queue state.

3.4 Design

3.4.1 Activity Diagram

The activity starts from a video source (local file, webcam, or RTSP stream), applies person detection, enriches detections with tracker IDs, then evaluates polygon-zone

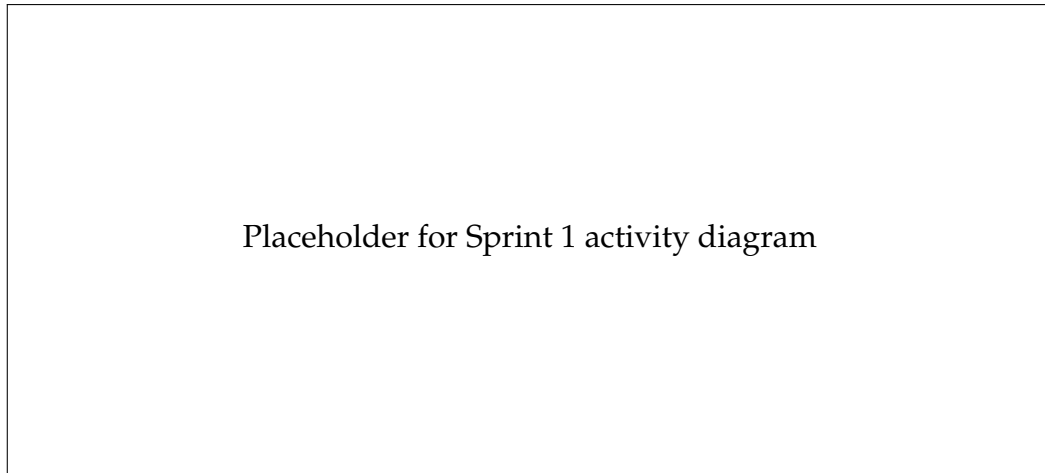


Figure 3.2: Sprint 1 activity flow from frame ingestion to queue-zone updates.

membership. The filtered state is forwarded to the queue analyzer for rate and wait-time computation.

3.4.2 Sequence Diagrams

The implemented sequence in the codebase is: `VideoStream/RTSPCamera` $\rightarrow$ `PersonDetector.detect()` $\rightarrow$ `ObjectTracker.update()` $\rightarrow$ `ZoneManager.trigger()` $\rightarrow$ `QueueAnalyzer.update()`.

This call chain is orchestrated per frame by `run()` in `backend/src/main.py`.

3.5 Implementation

3.5.1 Detection Layer

Detection is implemented in `backend/src/detector.py` through the `PersonDetector` class. It wraps Ultralytics YOLO inference and filters results to person class only (COCO class ID 0). The detector resolves device selection (`auto/cpu/cuda/mps`) and supports configurable image size for throughput control [2].

3.5.2 Tracking Layer

Tracking is implemented in `backend/src/tracker.py` with `supervision.ByteTrack`. The tracker propagates persistent `tracker_id` values across frames, which is essential for counting arrivals and departures rather than raw per-frame detections [3].

3.5.3 Zone-Aware Filtering

Zone logic is implemented in `backend/src/zone_manager.py`. The module accepts polygons in normalized or pixel coordinates, auto-scales normalized points to frame size, and evaluates inclusion using `PolygonZone` with bottom-center anchors for robust queue-ground contact checks.

3.5.4 Main Loop Integration

The pipeline integration is implemented in `backend/src/main.py`. The runtime supports:

- Source abstraction via `open_video_source()` for file/webcam/RTSP.
- Process stride control through `-process-every-n-frames`.
- Real-time overlay rendering (detections, IDs, zone, and metrics).
- Headless mode and dashboard event emission hooks for future API/runtime integration.

This implementation produced a stable perception baseline for later queue analytics and uncertainty modeling [4].

3.6 Conclusion

Sprint 1 delivered the operational perception core: reliable person detection, ID tracking continuity, and queue-zone filtering. This foundation directly enabled Sprint 2, where statistical queue metrics and uncertainty quantification were added on top of these stable perception outputs.

Table of Content

4.1	Introduction	31
4.2	Sprint Backlog	31
4.3	Design	31
4.3.1	Component Diagram	31
4.3.2	Data Flow Diagram	31
4.3.3	Sequence Diagrams	32
4.4	Implementation	33
4.4.1	Queue Event Windows and Rate Estimation	33
4.4.2	Wait-Time Logic and Stability Handling	33
4.4.3	Bayesian and Variance-Based Uncertainty	34
4.4.4	Uncertainty Classification and Runtime Contract	34
4.5	Conclusion	34

4.1 Introduction

Sprint 2 transformed perception outputs into operational queue indicators. The focus moved from “who is detected” to “how the queue behaves” through arrival/service-rate estimation, waiting-time computation, and explicit uncertainty quantification.

4.2 Sprint Backlog

Sprint 2 backlog priorities were:

- Implement event-based queue rate estimation from tracked IDs in zone.
- Compute wait-time estimates with explicit stable/unstable queue handling.
- Add Bayesian credible intervals for rate metrics.
- Add wait-time uncertainty bounds and uncertainty-level classification.
- Extend runtime payloads and overlays with confidence information.

4.3 Design

4.3.1 Component Diagram

At component level, `QueueAnalyzer` receives in-zone tracker IDs and produces `QueueMetrics`, while `uncertainty.py` computes uncertainty estimates and confidence intervals consumed by both logs and dashboard payloads.

4.3.2 Data Flow Diagram

Data flow is event-centric: per-frame ID transitions produce arrival/departure events, event windows produce rates (λ, μ) , rates and occupancy produce wait-time estimates, and uncertainty modules provide interval bounds and level labels.

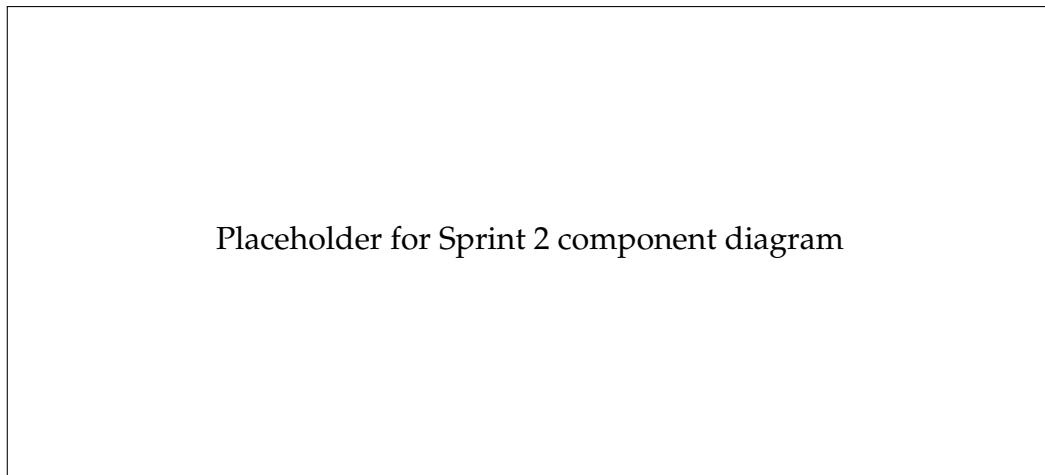


Figure 4.1: Sprint 2 components for queue analysis and uncertainty computation.

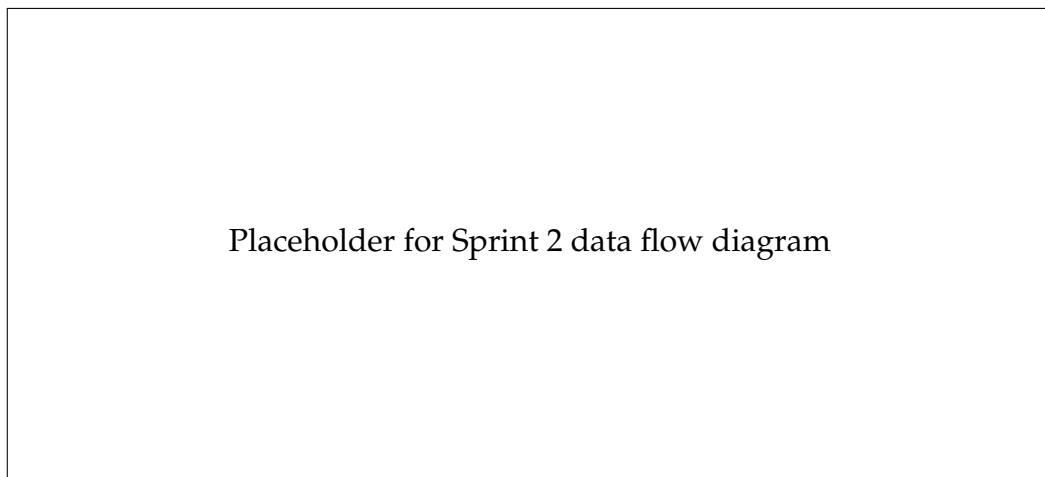


Figure 4.2: Data flow from tracked events to queue metrics and confidence intervals.

4.3.3 Sequence Diagrams

The implemented sequence is: `QueueAnalyzer.update()` computes counts and rates, then calls `estimate_rate_uncertainty()` and `estimate_wait_time_uncertainty_from_variance()`, then classifies uncertainty with `classify_uncertainty_level()`.

4.4 Implementation

4.4.1 Queue Event Windows and Rate Estimation

In `backend/src/queue_analyzer.py`, arrivals and departures are tracked as timestamp dequeues over sliding windows. Let N_a and N_d be events in windows T_a and T_d :

$$\lambda = \frac{N_a}{T_a}, \quad \mu = \frac{N_d}{T_d}$$

The implementation enforces a minimum event count before returning non-zero rates, reducing unstable early-window estimates.

4.4.2 Wait-Time Logic and Stability Handling

The runtime marks queue stability with:

$$\text{queue_stable} = (\lambda < \mu) \wedge (\mu > 0)$$

The wait-time logic follows two branches:

- Stable queue: $W = \frac{1}{\mu - \lambda}$
- Unstable but service observed: fallback $W = \frac{L}{\mu}$, where L is people in zone

When no service events are observed, the implementation currently returns a zero fallback.

4.4.3 Bayesian and Variance-Based Uncertainty

In `backend/src/uncertainty.py`, rate uncertainty uses a Gamma posterior with conjugate Poisson assumptions [5]:

$$\lambda \mid k, T \sim \Gamma(k + 1, T), \quad \mu \mid n, T \sim \Gamma(n + 1, T)$$

Credible intervals are computed via Gamma quantiles. Wait-time uncertainty is estimated from recent variance when history is sufficient; otherwise a controlled fallback margin is used.

4.4.4 Uncertainty Classification and Runtime Contract

Uncertainty is mapped to Low/Medium/High based on relative interval width. The final `QueueMetrics` object includes:

- point estimates (λ, μ, W) ,
- interval bounds for rates and wait time,
- stability flag and uncertainty-level label.

These fields are propagated to logs, overlays, webhook payloads, and dashboard events.

4.5 Conclusion

Sprint 2 delivered the analytical intelligence of the project: event-to-rate modeling, stable/unstable wait-time semantics, and uncertainty-aware outputs. This provided the mathematical and statistical core required for API exposure and external automation in the next sprint.

Table of Content

5.1	Introduction	36
5.2	Sprint Backlog	36
5.3	Design	36
5.3.1	Component Diagram	36
5.3.2	Data Flow Diagram	37
5.3.3	Sequence Diagrams	37
5.4	Implementation	38
5.4.1	RTSP Source Integration	38
5.4.2	ONVIF Discovery and Stream Resolution	38
5.4.3	FastAPI BFF and Typed Contract	38
5.4.4	Runtime Orchestration and Real-Time Events	39
5.4.5	Persistence and Recovery	39
5.5	Conclusion	39

5.1 Introduction

Sprint 3 extended the system from a single local pipeline to a connected multi-source platform. The work introduced robust RTSP and ONVIF camera handling, a typed FastAPI backend-for-frontend (BFF), and runtime services for feed lifecycle and real-time event distribution.

5.2 Sprint Backlog

The Sprint 3 backlog addressed four integration tracks:

- RTSP connectivity with authentication, transport control, reconnection, and snapshots.
- ONVIF discovery and stream resolution for IP camera onboarding.
- REST API endpoints for feed CRUD, metadata management, and diagnostics.
- Real-time delivery through websocket events and MJPEG feed streaming.

5.3 Design

5.3.1 Component Diagram

The design separates responsibilities between camera adapters (RTSP/ONVIF), API contract models, runtime feed orchestration, and websocket broadcasting. This modularization allows independent evolution of source onboarding, analysis workers, and frontend-facing interfaces.

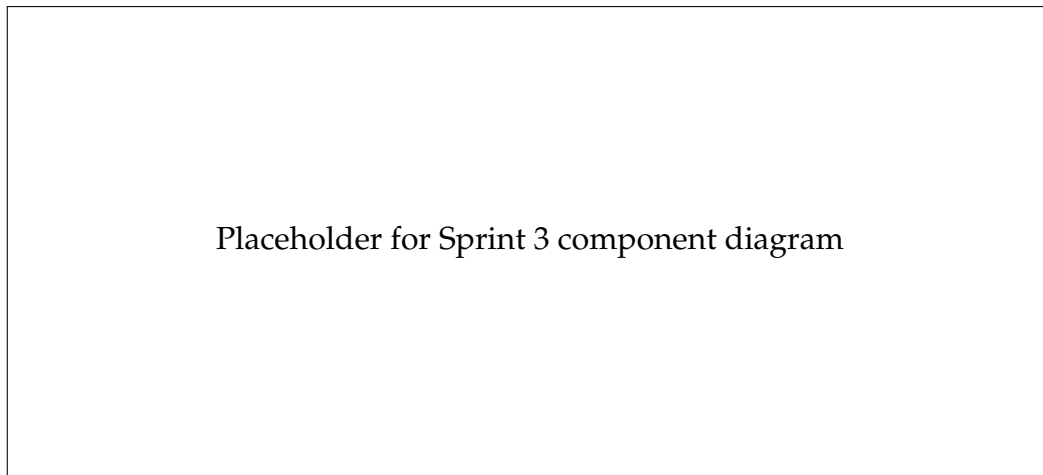


Figure 5.1: Sprint 3 components for camera integration and backend services.

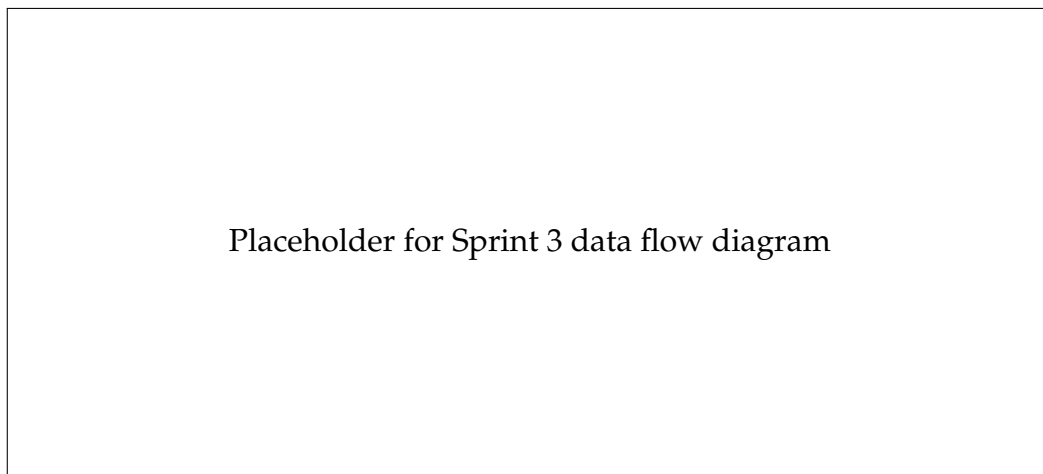


Figure 5.2: Data flow from video feeds to API responses and WebSocket broadcasts.

5.3.2 Data Flow Diagram

The principal data paths are: (1) onboarding/test flows through REST endpoints, (2) feed-control flows through start/stop/restart operations, and (3) runtime event flows from worker JSONL streams to websocket broadcast and dashboard cache updates.

5.3.3 Sequence Diagrams

The implemented sequence for a running feed is: frontend request → `/api/feeds/*` →

`FeedRegistry` operation → worker process lifecycle change → `websocket feed_status/metrics_update` event.

5.4 Implementation

5.4.1 RTSP Source Integration

In `backend/src/rtsp_camera.py`, the `RTSPCamera` class supports:

- credential embedding with safe log redaction,
- TCP/UDP transport selection through OpenCV/FFMPEG options,
- connection testing and snapshot capture,
- automatic reconnection with bounded retries.

The source abstraction in `backend/src/video_capture.py` routes local/webcam sources to `VideoStream` and RTSP URLs to `RTSPCamera`.

5.4.2 ONVIF Discovery and Stream Resolution

`backend/src/onvif_client.py` implements WS-Discovery probes, `ProbeMatch` parsing, capability extraction, media-service resolution, and RTSP URI retrieval through SOAP requests with digest/basic authentication fallback. This enables dynamic IP camera onboarding without hardcoding stream URLs.

5.4.3 FastAPI BFF and Typed Contract

The API layer is implemented in `backend/src/api/app.py`, with typed models in `backend/src/api/models.py`. Major endpoint groups include:

- feed management (create/list/start/stop/restart/delete),
- source onboarding (RTSP test/snapshot, ONVIF discovery/streams/test),
- metadata (establishments and caisses),
- integration and health (webhook status/test, system health),

- live channels (websocket and MJPEG stream endpoint).

5.4.4 Runtime Orchestration and Real-Time Events

Runtime control resides in `backend/src/api/runtime.py`. `FeedRegistry` manages feed state, persistence sync, worker process lifecycle, and event dispatch. The worker command launches `src.main` in headless mode and writes event JSONL. `WebSocketHub` broadcasts standardized events: `snapshot`, `feed_status`, `metrics_update`, `alert_fired`, and `system_warning`.

For video surfaces, `FeedFrameStreamManager` supports shared MJPEG buffering with capture-to-worker handoff continuity, preventing stream discontinuities when runtime mode changes.

5.4.5 Persistence and Recovery

`backend/src/database.py` provides SQLite persistence for feed configuration, metadata, video sessions, and archived alerts. On API startup, persisted feeds are restored and stale running sessions are safely recovered to a stopped state.

5.5 Conclusion

Sprint 3 delivered the system connectivity layer and formal service contract required by the web application. With source onboarding, runtime orchestration, and typed API/websocket channels in place, the project was ready for full web UI integration and operational workflows.

Table of Content

6.1	Introduction	41
6.2	Sprint Backlog	41
6.3	Design	41
6.3.1	UI and UX Design	41
6.3.2	Frontend Architecture	42
6.4	Implementation	42
6.5	Conclusion	43

6.1 Introduction

Sprint 4 defined the visual language and structural foundations of the web application. The objective was to move from backend-centric tooling to a coherent dashboard-ready frontend architecture, while preserving traceability with the queue analytics domain.

6.2 Sprint Backlog

Sprint 4 backlog focused on design-to-code readiness:

- Define a consistent visual design system (colors, typography, spacing, component rules).
- Produce complete page layouts for onboarding, monitoring, editing, analytics, and settings.
- Scaffold a React + TypeScript frontend project with reusable UI primitives.
- Establish routing and layout composition patterns for future live integration.

6.3 Design

6.3.1 UI and UX Design

The web design specification in `design/DESIGN_SPEC.md` defines a “Terminal Minimal” style system and a seven-page navigation model. The design language emphasizes:

- dark operational surfaces optimized for continuous monitoring,
- strong semantic colors for status and alert levels,
- card-based information density with consistent spacing and hierarchy,
- explicit workflow guidance in setup and zone-editing screens.

The designed page set includes landing, dashboard, setup wizard, zone editor, analytics, settings, and heatmap views, which directly informed component boundaries in the frontend implementation.

6.3.2 Frontend Architecture

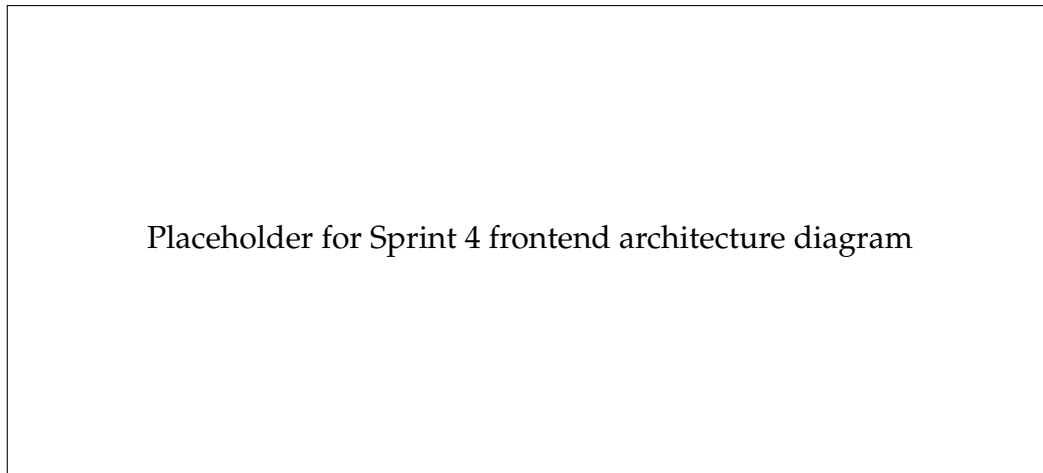


Figure 6.1: Frontend architecture for routing, components, and state boundaries.

The scaffolded frontend architecture separates page-level orchestration, shared components, hooks, and API adapters:

- `frontend/src/pages`: route-level views (Dashboard, Analytics, ZoneEditor, Settings, etc.).
- `frontend/src/components`: reusable UI and domain panels.
- `frontend/src/hooks`: stateful integration and workflow hooks.
- `frontend/src/lib`: API contract and helper utilities.

6.4 Implementation

Sprint 4 implementation established the frontend technical baseline:

- Vite + React + TypeScript project initialization (`frontend/package.json`).

- Tailwind CSS and shadcn/Radix-driven UI composition.
- React Router page structure and layout shell integration.
- Initial dashboard and setup page scaffolding for subsequent live-data wiring.

This phase intentionally prioritized architecture and design consistency over full real-time behavior. It produced the stable structure later used by Sprint 5 for websocket and API integration.

6.5 Conclusion

Sprint 4 delivered a complete design-to-frontend scaffold transition: the visual specification was formalized and mapped to a maintainable React architecture. This reduced integration risk for Sprint 5, where live backend data and runtime controls were connected.

Sprint 5: Frontend Integration and Live Monitoring

Table of Content

7.1	Introduction	45
7.2	Sprint Backlog	45
7.3	Design	45
7.3.1	Data Flow Diagram	45
7.3.2	Sequence Diagrams	45
7.4	Implementation	46
7.4.1	Typed API Contract Integration	46
7.4.2	Dashboard State Orchestration	46
7.4.3	WebSocket Reliability and Fallback Logic	47
7.4.4	Dashboard UI Integration	47
7.4.5	Integration Testing	47
7.5	Conclusion	48

7.1 Introduction

Sprint 5 connected the frontend foundation to real backend services and runtime events. The objective was to provide an operational live dashboard capable of feed control, stream visualization, metric updates, and alert awareness.

7.2 Sprint Backlog

Sprint 5 backlog covered integration-critical items:

- implement a typed API client aligned with backend response contracts,
- integrate feed lifecycle actions (create/start/stop/restart/delete),
- integrate websocket events for low-latency dashboard updates,
- implement resilient live monitoring behavior (reconnect and fallback),
- validate dashboard setup and onboarding workflows through integration tests.

7.3 Design

7.3.1 Data Flow Diagram

The data contract follows a layered flow: REST endpoints provide CRUD and diagnostics, websocket events provide live updates, and local dashboard state aggregates feed metrics into KPI cards, activity logs, and attention panels.

7.3.2 Sequence Diagrams

Three primary sequences were implemented:

1. **Feed lifecycle:** dashboard action → REST endpoint → registry update → websocket `feed_status` event.

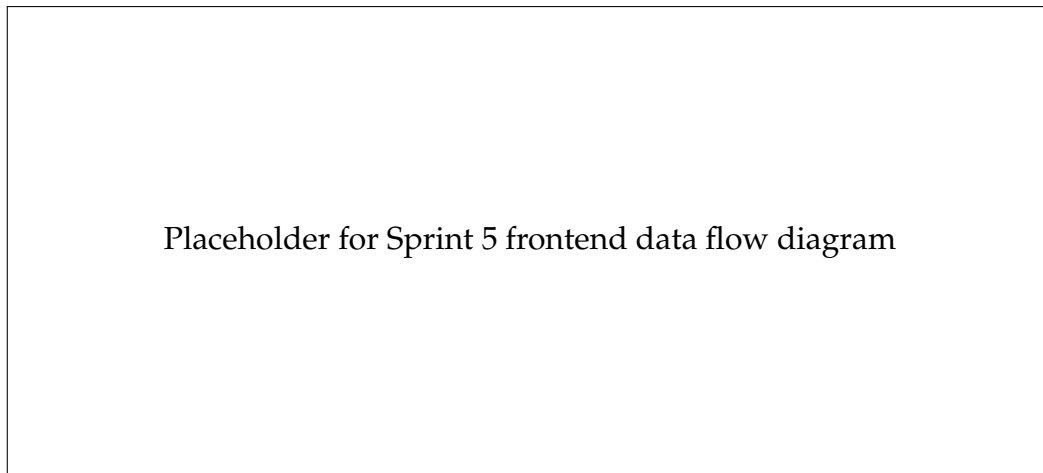


Figure 7.1: Data flow between backend services and frontend monitoring components.

2. **Live metrics:** worker metrics event → backend websocket → frontend cache merge → visual refresh.
3. **Alert surfacing:** backend alert event → websocket alert_fired/system_warning → activity and attention views.

7.4 Implementation

7.4.1 Typed API Contract Integration

The frontend contract is implemented in `frontend/src/lib/api.ts`. It defines typed entities for feeds, metrics, alerts, and websocket events, and wraps network calls with explicit error classes (`ApiError`, `NetworkError`, `ValidationError`). This improved reliability of frontend-side error handling and debugging.

7.4.2 Dashboard State Orchestration

`frontend/src/hooks/use-live-dashboard.ts` coordinates data fetching and mutations through React Query. Derived indicators (online feeds, total people, average wait time, attention counts) are computed from normalized feed state and exposed to dashboard widgets.

7.4.3 WebSocket Reliability and Fallback Logic

`frontend/src/hooks/use-dashboard-websocket.ts` implements:

- reconnect backoff,
- heartbeat ping,
- stale-data watchdog,
- REST polling fallback during websocket degradation,
- batched metric flush to reduce UI re-render pressure.

This design ensures graceful degradation under transient connection issues.

7.4.4 Dashboard UI Integration

`frontend/src/pages/Dashboard.tsx` integrates live feed controls, setup flow dialogs, source onboarding (RTSP/ONVIF/local file), KPI visualizations, and chart components. The page composes domain hooks and reusable dashboard components into one operational interface.

7.4.5 Integration Testing

Frontend integration behavior is validated with Vitest and Testing Library in:

- `frontend/src/pages/Dashboard.integration.test.tsx`,
- `frontend/src/hooks/use-live-dashboard.test.tsx`,
- `frontend/src/lib/api.test.ts`.

These tests cover setup flow execution, websocket-event handling, and API error contract behavior.

7.5 Conclusion

Sprint 5 delivered a fully connected monitoring frontend: typed REST integration, resilient websocket updates, and operational dashboard controls. This integration created the interface layer required for end-to-end alert automation in Sprint 6.

Table of Content

8.1	Introduction	50
8.2	Sprint Backlog	50
8.3	Design	50
8.3.1	Component Diagram	50
8.3.2	Data Flow Diagram	50
8.4	Implementation	51
8.4.1	Webhook Payload Contract	51
8.4.2	Reliable Webhook Delivery	52
8.4.3	Runtime Alert Dispatch Integration	52
8.4.4	n8n Workflow Routing and Telegram Delivery	52
8.4.5	Automation Testing Artifacts	53
8.5	Conclusion	53

8.1 Introduction

Sprint 6 integrated the queue analytics pipeline with external automation workflows through n8n. The objective was to transform runtime metrics into actionable alerts with transport reliability, cooldown control, and optional Telegram notification delivery.

8.2 Sprint Backlog

Sprint 6 backlog focused on operational automation:

- define and validate a stable webhook payload contract,
- implement robust webhook transmission with retries and backup logging,
- integrate alert triggers from runtime threshold detection,
- implement n8n routing for authorization, cooldown, formatting, and delivery,
- provide test scripts for repeated workflow validation.

8.3 Design

8.3.1 Component Diagram

The automation design includes five components: queue runtime metrics, webhook payload serializer, HTTP delivery client, n8n workflow router, and notification endpoints. Alert archival is additionally routed back to backend through a dedicated API endpoint.

8.3.2 Data Flow Diagram

Data flow starts with periodic metrics dispatch from the runtime loop, branches on alert presence and cooldown in n8n, and then either sends Telegram notifications or returns

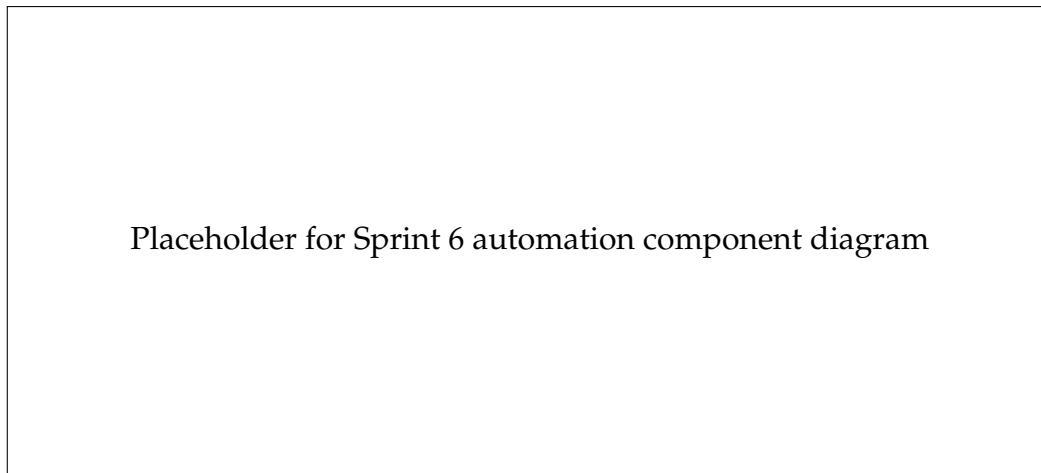


Figure 8.1: Components involved in backend-to-n8n-to-Telegram automation flow.

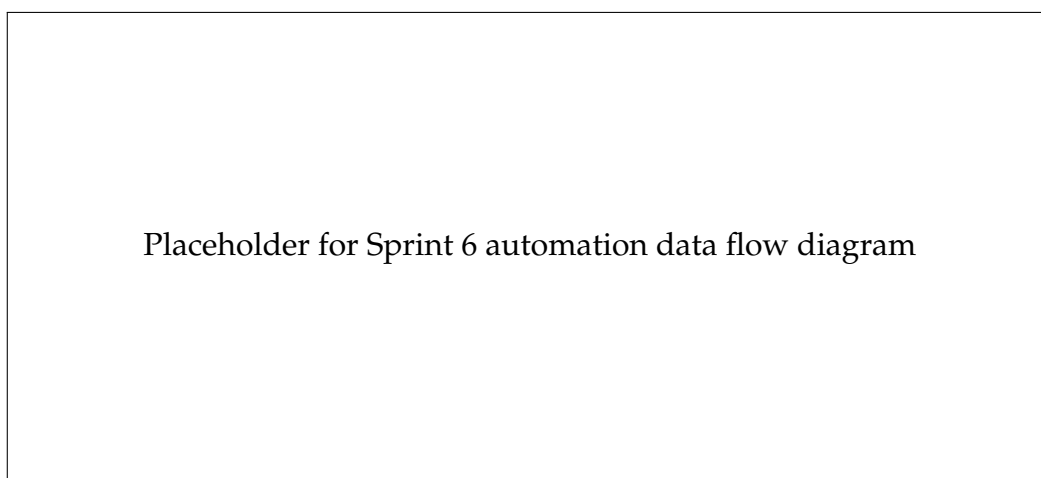


Figure 8.2: Alert payload flow from backend metrics to external notifications.

acknowledgment without external dispatch. Archived alert payloads are persisted through `/api/alerts/archive`.

8.4 Implementation

8.4.1 Webhook Payload Contract

`backend/src/webhook.py` defines `QueuePayload`, a structured contract carrying:

- source metadata (timestamp, frame, source, feed ID),
- queue metrics (λ , μ , wait time, stability),

- uncertainty bounds and classification,
- alert flags and severity fields.

The module includes payload validation before dispatch.

8.4.2 Reliable Webhook Delivery

`backend/src/webhook_client.py` sends payloads to n8n with timeout, retry policy, optional shared-secret header, and success/failure statistics. Each payload is also written to local CSV through `CSVMetricsLogger` as a backup persistence path.

8.4.3 Runtime Alert Dispatch Integration

In `backend/src/main.py`, webhook sending is offloaded to `AsyncWebhookDispatcher` to avoid blocking the main perception loop. The implementation supports periodic metric dispatch and immediate warning dispatch with deduplication windows to limit repeated alert spam.

8.4.4 n8n Workflow Routing and Telegram Delivery

The workflow file `n8n_workflow_with_telegram.json` implements:

- webhook reception and secret validation,
- alert detection and cooldown gating,
- alert history archival via backend API,
- message formatting and Telegram send node,
- explicit acknowledge/reject branches.

This structure provides auditable routing between no-alert, alert-suppressed, and alert-delivered branches.

8.4.5 Automation Testing Artifacts

Workflow validation scripts are provided in `scripts/` (e.g., `test_n8n_send_alert.ps1`, `test_n8n_automated.ps1`, `test_n8n_multi_feed_scenario.ps1`). These scripts automate webhook submission, optional execution polling, and branch validation scenarios.

8.5 Conclusion

Sprint 6 operationalized the analytics output through a robust automation path. With validated payload contracts, resilient transport, and n8n-driven routing, queue events are now consumable by external alerting channels and archival workflows.

Sprint 7: Evaluation and Optimization

Table of Content

9.1	Introduction	55
9.2	Sprint Backlog	55
9.3	Design	55
9.3.1	Test Plan	55
9.3.2	Performance Metrics	56
9.4	Implementation	56
9.4.1	Automated Test Coverage	56
9.4.2	Runtime Optimization Actions	56
9.4.3	Resilience Improvements	57
9.4.4	Observed Outcome	57
9.5	Conclusion	57

9.1 Introduction

Sprint 7 focused on validation and runtime optimization of the end-to-end stack. The work combined unit and integration tests, operational hardening, and throughput tuning for both backend runtime and dashboard experience.

9.2 Sprint Backlog

Sprint 7 backlog included:

- consolidate backend and frontend automated test coverage,
- validate feed lifecycle, stream continuity, and API contracts,
- optimize runtime throughput and responsiveness,
- harden failure handling (network drops, stale data, worker exits),
- document residual limitations for final delivery planning.

9.3 Design

9.3.1 Test Plan

Validation followed a layered strategy:

- **Backend unit/integration tests:** uncertainty, API endpoints, runtime streaming behavior, webhook payload handling.
- **Frontend integration tests:** dashboard setup flow, websocket event handling, and API error handling.
- **Scenario tests:** n8n send/acknowledge flows, unauthorized branch, and multi-feed alert scenarios.

9.3.2 Performance Metrics

The optimization metrics were aligned with operational requirements:

- processing responsiveness (loop cadence and processed-frame throughput),
- stream continuity under feed state transitions,
- webhook impact on main-loop latency,
- dashboard freshness under websocket degradation.

9.4 Implementation

9.4.1 Automated Test Coverage

Representative test assets include:

- backend: `backend/tests/test_api_app.py`, `test_runtime_streams.py`, `test_webhook.py`,
- backend/root: `test_uncertainty.py`,
- frontend: `frontend/src/pages/Dashboard.integration.test.tsx`, `frontend/src/hooks/use`
`frontend/src/lib/api.test.ts`.

These tests verify API envelopes, event contracts, stream handoff, uncertainty calculations, and setup-workflow behavior.

9.4.2 Runtime Optimization Actions

Key optimizations implemented in `backend/src/main.py` and `backend/src/config.py` include:

- configurable detector image size and frame stride,
- explicit inference device control and diagnostics,
- asynchronous webhook dispatch to keep the processing loop responsive,

- real-time playback pacing for file sources.

9.4.3 Resilience Improvements

Runtime resilience was improved at multiple levels:

- feed worker supervision and exit recovery in `FeedRegistry`,
- capture-to-worker frame handoff continuity in `FeedFrameStreamManager`,
- websocket reconnect/fallback logic on the frontend side.

9.4.4 Observed Outcome

The system reached a stable integration state where perception, analytics, API runtime, dashboard, and n8n automation operate as a coherent pipeline. Remaining work is mostly focused on extended benchmarking campaigns and production-grade deployment hardening rather than missing core functionality.

9.5 Conclusion

Sprint 7 consolidated quality and readiness through structured testing and targeted performance improvements. The resulting platform is functionally complete for demonstration and academic evaluation, with clearly identified extensions for industrial-scale rollout.

Table of Content

10.1 Summary	59
10.2 Objective Review	59
10.3 Future Work	60

10.1 Summary

This end-of-studies project delivered a complete queue wait-time estimation platform that connects computer vision, queue analytics, uncertainty quantification, web supervision, and automation workflows.

From a technical standpoint, the system now includes:

- a perception pipeline based on YOLO detection and ByteTrack identity continuity,
- queue-zone filtering and event-driven estimation of λ , μ , and expected wait time,
- uncertainty-aware outputs with interval bounds and stability semantics,
- a typed FastAPI backend-for-frontend with feed lifecycle control and live web-socket events,
- a React dashboard for monitoring, setup, and runtime interaction,
- n8n and Telegram integration for operational alert routing.

The resulting architecture demonstrates that a full monitoring-to-action loop can be achieved with modular components while keeping the system maintainable and testable.

10.2 Objective Review

The initial objectives are considered achieved at project scope level:

1. **Real-time detection and tracking:** implemented through dedicated detector and tracker modules.
2. **Queue-zone counting:** implemented with polygon-zone management and in-zone identity filtering.

3. **Queue metric estimation:** implemented for arrival rate, service rate, and wait time.
4. **Uncertainty quantification:** implemented with Bayesian and variance-based methods.
5. **Web monitoring:** implemented through a multi-feed dashboard with live updates.
6. **Automation integration:** implemented through webhook transport, n8n routing, and Telegram notifications.

The main remaining gap is not architectural completeness, but depth of long-horizon evaluation under broader operational scenarios.

10.3 Future Work

Potential extensions include:

- multi-zone and multi-server queue modeling (for parallel cashier configurations),
- richer alert strategies (cooldown tuning, escalation policies, acknowledgment loops),
- edge deployment optimization (ONNX/TensorRT profiling and resource-aware scheduling),
- long-duration field evaluation with domain-specific KPI baselines,
- stronger security controls for production webhook exposure and secrets management.



References

- [1] D. Gross, J. F. Shortle, J. M. Thompson, and C. M. Harris, *Fundamentals of Queueing Theory*, 5th ed. Wiley, 2018.
- [2] Ultralytics, *Ultralytics documentation*, <https://docs.ultralytics.com/>, 2026.
- [3] Y. Zhang et al., “Bytetrack: Multi-object tracking by associating every detection box,” *arXiv preprint arXiv:2110.06864*, 2022.
- [4] OpenCV, *Opencv documentation*, <https://docs.opencv.org/>, 2026.
- [5] C. M. Bishop, *Pattern Recognition and Machine Learning*. Springer, 2006.



A.1 Useful Commands

- Run backend API: `uvicorn backend.app:app -reload`
- Run backend tests: `pytest tests -q`
- Run frontend dev server: `cd frontend && npm run dev`
- Run frontend tests: `cd frontend && npm run test`

A.2 Example Webhook Payload

```
{  
  "timestamp": "2026-03-05T14:32:10.123Z",  
  "camera_id": "cam_01",  
  "zone_id": "checkout_lane_3",  
  "metrics": {  
    "people_in_zone": 7,  
    "arrival_rate": 0.15,  
    "service_rate": 0.16,  
    "wait_time_seconds": 15.0,  
    "queue_stable": true  
  },  
  "uncertainty": {  
    "lambda_ci": [0.10, 0.22],  
  }  
}
```

```
"mu_ci": [0.11, 0.23],  
"wait_time_ci": [9.5, 24.3],  
"level": "MEDIUM"  
},  
"alerts": [  
  {  
    "type": "WAIT_TIME_WARNING",  
    "severity": "warning",  
    "message": "Wait time exceeded 60s threshold",  
    "value": 72.3,  
    "threshold": 60  
  }  
],  
"raw_detection_count": 7,  
"fps": 24.5  
}
```

A.3 Items to Add Before Final Submission

- final versions of all architecture and sequence diagrams,
- finalized evaluation tables and plots for Sprint 7,
- appendix references for API and test artifacts,
- proofreading pass for terminology and citation consistency.